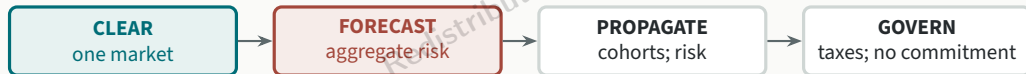


Session 4.2

Krusell-Smith, Solved Three Ways

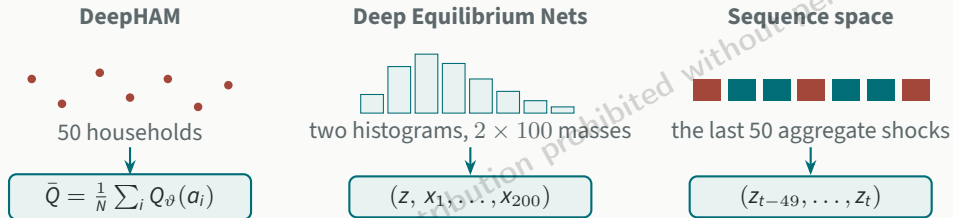
Generalised moments, a histogram, a history of shocks: three ways to put a moving distribution into a network, and what each can and cannot see



Zhigang Feng

The pages marked PRELIMINARY are based on preliminary research by Zhigang Feng, Jiequn Han, Thomas J. Sargent, and Shenghao Zhu (September 2026). Do not circulate without the authors' permission.

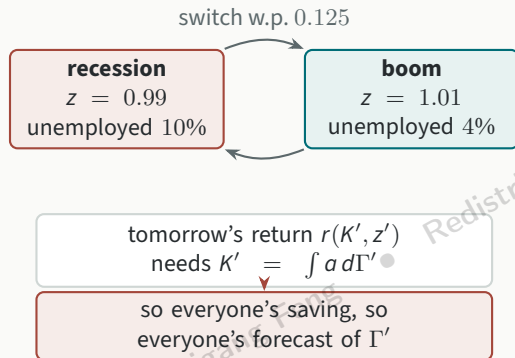
Three Ways to Give a Network a Moving Distribution



- Krusell and Smith's economy: the household's state is $(a, \varepsilon; z, \Gamma)$, and Γ is a distribution.
- Each replaces Γ by something finite. For each: what the network reads, what it minimises, *where its training distributions come from*.
- **Then Deck 4.1's method, which writes down how Γ moves.**

Source: Han, Yang & E, *QE* (2026); Azinovic, Gaegauf & Scheidegger, *IER* (2022); Azinovic-Yang & Žemlička (2025).

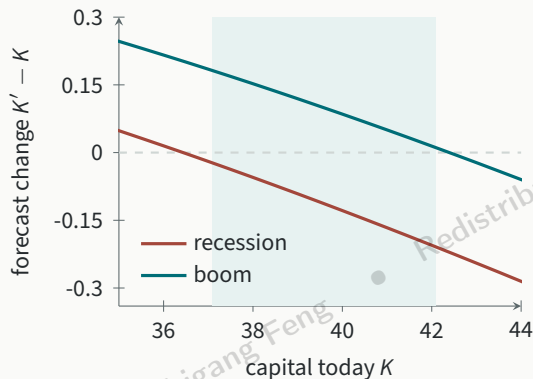
The Economy, and Why the Household Must Forecast Γ



- ▶ $V(a, \varepsilon; z, \Gamma) = \max_{a' \geq 0} \ln c + \beta \mathbb{E} V(a', \varepsilon'; z', \Gamma'),$
 $c + a' = (1 - \delta + r)a + y(\varepsilon).$
- ▶ r, w are marginal products at $K = \int a d\Gamma$; employment $\varepsilon \in \{0, 1\}$ moves with z .
- ▶ $\Gamma' = H(\Gamma, z, z')$: **the law of motion every household needs and nobody is given.**

Source: Krusell & Smith, *JPE* (1998); Den Haan, Judd & Juillard, *JEDC* (2010).

The Classical Answer: Forecast With the Mean



- ▶ The bet: $\log K' = \kappa_0(z) + \kappa_1(z) \log K$. The state shrinks to (a, ε, K, z) .
- ▶ Solve at the rule, push a histogram 3,000 quarters, regress, update: 17 CPU-seconds.
- ▶ $R^2 > 0.99999$ one step ahead. Run alone for 2,000 quarters: mean miss 0.054%, worst 0.187%.
- ▶ **The benchmark, and the idea each deep method generalises.**

Source: Krusell & Smith (1998); Den Haan, *JEDC* (2010); the lab, block B. Quarterly, as published: $\alpha = 0.36, \beta = 0.99, \delta = 0.025, u = \ln c, z \in \{0.99, 1.01\}$.

Four Questions Every Deep Method Answers

	DeepHAM	Deep Equilibrium Nets	Sequence space
1. what stands for Γ	learned moments of a 50-household panel	a 100-point histogram per employment state	the last 50 aggregate shocks
2. what is minimised	minus lifetime utility over a rollout; a value regression	Euler and KKT residuals	a Fischer–Burmeister Euler residual
3. where training states come from	simulated panels, refreshed	16 simulated economies, carried forward	a replay buffer of simulated histories
4. what the gradient passes through	150 simulated periods of the panel	one period, at stored next states	one period; prices held fixed

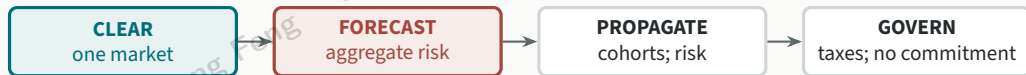
- **Row 3: in all three, training distributions come from simulating the network's own policy.**

Source: DeepHAM's public repository; the Deep Equilibrium Nets continuum-of-agents code; the sequence-space tutorial.

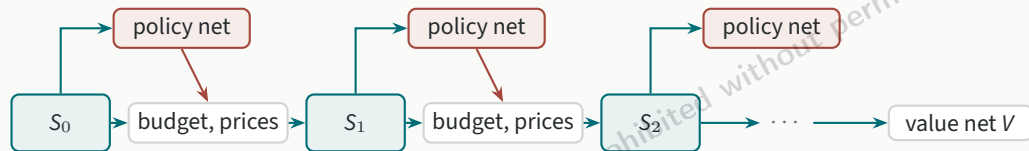
Session 4.2

Method 1: DeepHAM

A network inside an average learns the moments that matter, and the policy is trained on simulated paths



DeepHAM in One Picture: an Unrolled Panel

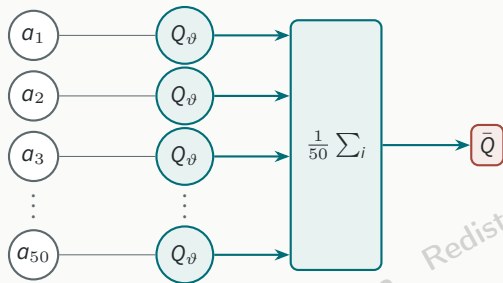


S_t : all 50 households' (a_i, ε_i) and z_t ; prices from $K_t = \frac{1}{50} \sum_i a_i$

- ▶ 384 economies run side by side, each a panel of $N = 50$ households.
- ▶ The actor simulates 150 periods forward from a stored cross-section inside the loss; a critic values what is left. Where the stored cross-sections come from: the training-loop pages.
- ▶ **Both networks read the panel through generalised moments.**

Source: Method 1 follows Han, Yang & E, "DeepHAM: A Global Solution Method for Heterogeneous Agent Models with Aggregate Shocks", *QE* 17(2), 2026 (preprint arXiv 2112.14377), and its public repository (Figure 2, §3).

Generalised Moments: a Network Inside an Average

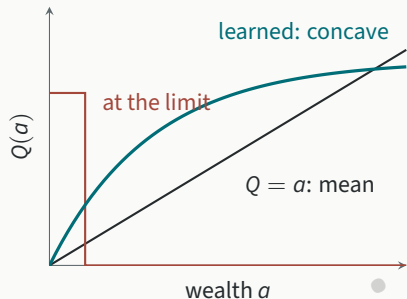


one network, shared by all:
2 layers of 12 tanh units

- ▶ $\bar{Q}_t = \frac{1}{N} \sum_i Q_{\vartheta}(a_t^i)$: one small network, applied to each household's wealth, then averaged.
- ▶ Household i 's policy reads $(a_i, \varepsilon_i, z_t, \bar{Q}_t)$.
- ▶ ϑ is trained with the policy and value networks: the moment is whatever lowers the loss.
- ▶ **Krusell-Smith run: $J = 1$, one generalised moment of wealth.**

Source: The paper, §2.2; repository *util.py* and its Krusell-Smith configuration.

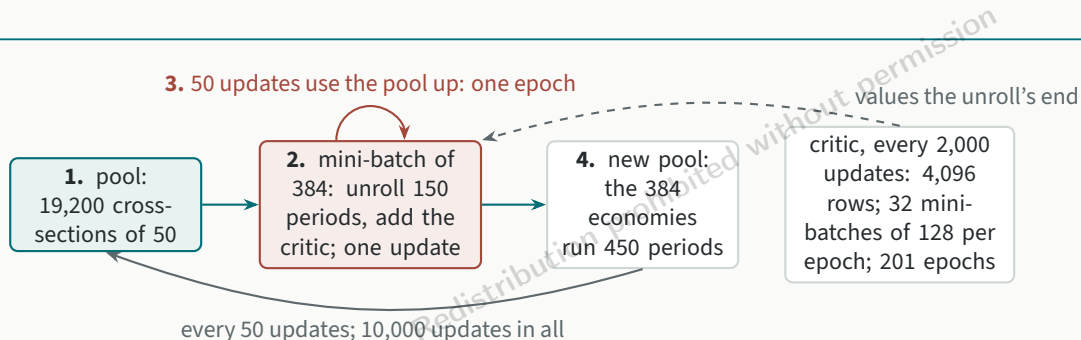
Why an Average, and What It Nests



- ▶ **Permutation invariance:** renumbering households leaves $\sum_i Q(a_i)$ unchanged.
- ▶ $Q(a) = a$ gives the mean, Krusell and Smith's state; an indicator of the limit gives the hand-to-mouth share.
- ▶ Learned here: Q concave in wealth, value linear in $\bar{Q}$. Moving wealth from rich to poor raises $\bar{Q}$ at the same K .
- ▶ **Bellman error: the Krusell–Smith rule 0.0253; DeepHAM with the mean 0.0184; with one generalised moment 0.0151.**

Source: The paper, §3.2, Table 2, Figure 3. Curves schematic.

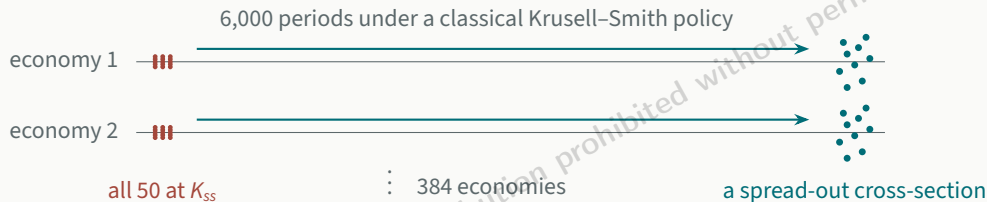
DeepHAM's Training Loop in One Picture



- ▶ **1. Samples:** a pool of 19,200 cross-sections of 50, from the 384 burned-in economies.
- ▶ **2.** A mini-batch is 384 of them.
- ▶ **3–4.** 50 updates use the pool up; then a new pool. The critic refits in its own loop.
- ▶ **The 150-period unroll is inside the loss: it moves nothing in the pool.**

Source: Repository *train_KS.py*, *policy.py* (*train*, *sampler*), *dataset.py*; the Krusell–Smith configuration.

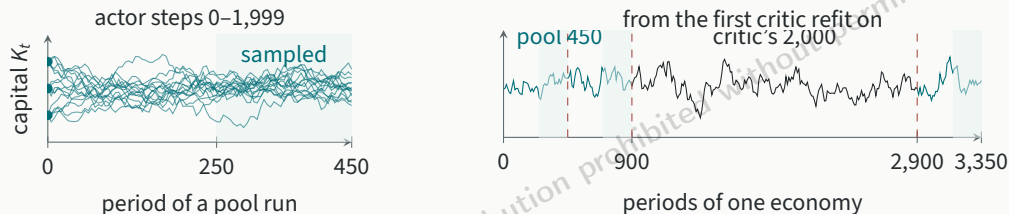
1. Samples: 384 Economies of 50, Burned In



- ▶ 384 separate economies run side by side, each with 50 households and its own aggregate shocks.
- ▶ Each starts with everyone at the representative-agent capital; employment drawn at the aggregate state's unemployment rate.
- ▶ After 6,000 periods the 384 final cross-sections are the **initial states**.
- ▶ **A “distribution” is a sample of 50 households; the algorithm holds 384 of them.**

Source: Repository *dataset.py* (*KSInitDataSet*), *simulation_KS.py*, the Krusell–Smith configuration.

1. Samples Over Training: 384 Tracks



- ▶ Left, actor steps 0-1,999: every pool re-run starts from the *same* burned-in states (6,000 periods from K_{ss}), on fresh shocks.
- ▶ Right, from the first critic refit: each re-run continues where the last ended; about 80,000 periods per economy by step 10,000.
- ▶ **Like Method 2: long simulations under the evolving policy; each datum is 50 households.**

Source: Repository `dataset.py` (`update_from_simul`), `policy.py` (`train`, `sampler`), the Krusell-Smith configuration (`update_init`). Paths schematic.

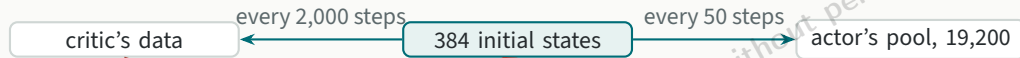
2-3. Actor: Mini-Batches of 384; the Pool Used Up in 50 Steps



- ▶ **Dataset:** the pool, 19,200 cross-sections from a 450-period run of the 384 economies.
- ▶ **Mini-batch:** the pool is shuffled once; each update takes the next 384 cross-sections as the starting states S_0 of its unroll.
- ▶ **Inside the loss:** from each S_0 , fresh shocks and 150 simulated periods; household 1's discounted utility is one sample. This does not move the pool.
- ▶ **Epoch:** $19,200/384 = 50$ updates use every snapshot once; then the pool is rebuilt, the next episode.

Source: Repository *dataset.py* (*get_policydataset*, *crazyshuffle*) and *policy.py* (*sampler*, *train*).

4. Episodes: Three Clocks, and the Algorithm



after the first refit, every re-simulation moves the initial states

DeepHAM, Krusell–Smith configuration

- 1 burn in 384 economies of 50 for 6,000 periods; fit critics under a constant consumption share
- 2 build the pool: 450 periods, 50 snapshots per economy
- 3 10,000 actor steps: 384 cross-sections from the pool, unroll 150, one Adam step
- 4 every 50 steps: re-simulate the pool; every 2,000: re-simulate the critic's data and refit

- **Every training distribution is a stretch of these 384 simulated economies, under the evolving policy.**

Source: Repository *train_KS.py*, *policy.py* (*train*, *sampler*), *dataset.py* (*update_from_simul*).

The Critic's Own Loop: Data, Mini-Batches, Epochs



- **Data:** 4,608 cross-sections of 50 (230,400 targets); 512 set aside to check the fit, 4,096 for training.
- **Rows:** each of the 384 economies runs 2,000 periods from its initial state under the current actor; its cross-sections at the 12 start dates are the rows.
- **Samples:** each household's target is its own realised 800-period discounted utility along that same path.
- **Mini-batches and epochs:** 128 shuffled rows per Adam step, so $4,096/128 = 32$ per epoch; 201 epochs; three critics. Rebuilt every 2,000 actor updates.

Source: Repository *dataset.py* (*get_valuedataset*, *process_vdatadict*), *value.py*; the Krusell–Smith configuration.

5a. Inside One Actor Update: the Loss

- **Unroll**: simulate 150 periods forward *inside the loss*, from each S_0 in the batch B , so the gradient runs back through every period:

$$\max_{\theta} \frac{1}{384} \sum_{S_0 \in B} \left[\sum_{t=0}^{148} \beta^t \ln c_t^1 + \beta^{149} \bar{V}(S_{149}) \right], \quad c_t^i = \sigma(\mathcal{C}_{\theta}(a_i, \varepsilon_i, z_t, \bar{Q}_t)) \cdot \text{wealth}_t^i.$$

- $\bar{V}$ is the average of the three critics; $\beta^{149} \approx 0.22$ leaves the rest of the future to them. Adam, rate 4×10^{-4} .
- Only household 1 optimises; the other 49 use the same network with the gradient stopped, and set prices and $\bar{Q}$: **fictitious play**.
- **Not 384 cross-sections $\times$ 50 households per step: the reshuffle only makes household 1 a random member. Averaging all 50 would be a planner's problem.**

Source: The paper, eq. (10), Algorithm 1; repository *policy.py* (*loss*, the “socialplanner” option).

5b. Inside One Critic Update: the Loss

- ▶ A mini-batch B is 128 training cross-sections (s, e) ; every household in each counts:

$$\mathcal{L}(\psi; B) = \frac{1}{128 \cdot 50} \sum_{(s,e) \in B} \sum_{i=1}^{50} \left[v_{\psi}(a_{i,s}^e, \varepsilon_{i,s}^e, z_s^e, \bar{Q}_s^{V,e}) - \hat{V}_{i,s}^e \right]^2, \quad \hat{V}_{i,s}^e = \sum_{\tau=0}^{799} \beta^{\tau} \ln c_{i,s+\tau}^e.$$

- ▶ One Adam step, rate 10^{-4} , per mini-batch: 32 per pass, about 6,400 per critic.
- ▶ A row keeps its 50 households together: each household's input includes $\bar{Q}_s^{V,e}$, the moment of its own cross-section.
- ▶ $\beta^{800} < 3.3 \times 10^{-4}$: the truncated sum stands in for the infinite one.
- ▶ **Monte Carlo evaluation, as in Session 3: each target is one realised future, and the regression averages the noise.**

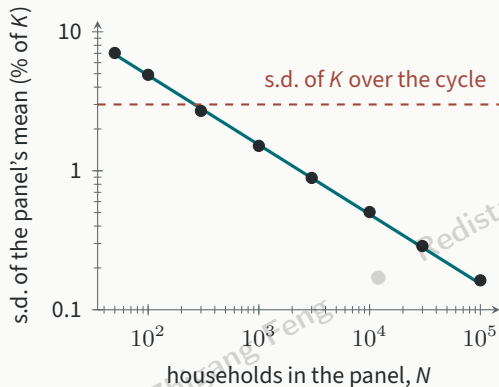
Source: The paper, eq. (8)–(9); repository *dataset.py*, *value.py*.

Ten Numbers, and What Each One Is

number	what it counts	why
50	households in one economy	the finite panel that stands for a continuum
384	economies side by side; the actor's batch	one draw of Γ each
6,000	periods of burn-in, once	from everyone at K_{ss} to a spread-out sample
2,000	periods simulated for the critic	12 start dates plus 800 periods after the last
100 / 800	spacing of the critic's start dates / length of each future	snapshots apart; β^{800} negligible
450 / 250 / 4	periods simulated for the actor / discarded / spacing of snapshots	adjust to the policy, then sample
19,200	cross-sections in the actor's pool	384 economies $\times$ 50 snapshots
150	periods in one actor rollout	$\beta^{149} \approx 0.22$ leaves the rest to the critic
128 / 6,400	critic mini-batch: cross-sections / household targets	one Adam step each; 32 per epoch
50 / 2,000	actor steps per pool / per critic refit	how often each dataset is rebuilt

Source: Repository Krusell–Smith configuration, *dataset.py*, *policy.py*.

What 50 Households Cost



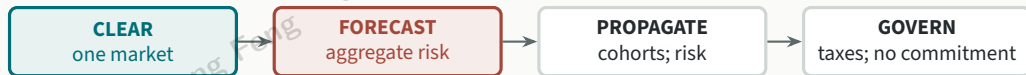
- ▶ Wealth has s.d. 18.9 around $K = 38.8$: a panel's mean misses K by $18.9/\sqrt{N}$.
- ▶ $N = 50$: 2.67, 2.3 times the s.d. of K over the whole cycle.
- ▶ The paper treats $N = 50$ as the model, with similar results at 100 and 200.
- ▶ **A sampled state, and prices that inherit its noise.**

Source: Lab block B, 400 panels per N . Quarterly, as published: $\alpha = 0.36, \beta = 0.99, \delta = 0.025, u = \ln c, z \in \{0.99, 1.01\}$. The paper, fn. 2.

Session 4.2

Method 2: Deep Equilibrium Nets

Keep the whole histogram, and ask only that the first-order conditions hold one period ahead



The Same Economy, With a Histogram as the State

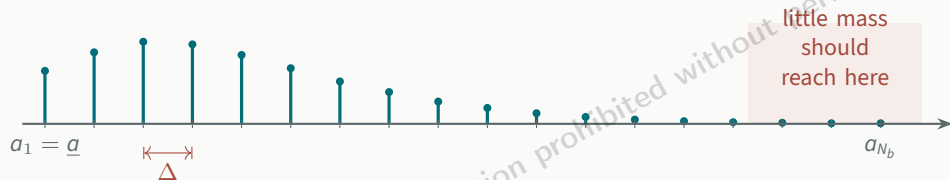


one asset grid of 100 points;
a histogram of the unemployed
and one of the employed

- ▶ State $X = (z, h_u, h_e)$: the shock, and two histograms on one asset grid: 2×100 masses.
- ▶ Two, because employment is persistent: who is poor today predicts who is unemployed tomorrow, so assets alone do not determine next period.
- ▶ One network: $(\varepsilon, a; X) \mapsto (a', \lambda)$. Prices follow from K .
- ▶ **Unlike DeepHAM: every mass is an input; no learned moment, no critics.**

Source: Method 2 follows Azinovic, Gaegauf & Scheidegger, “Deep Equilibrium Nets”, *IER* 63(4), 2022, and its public continuum-of-agents code, adapted to Krusell–Smith. Here: App. A.5; the code’s state. A mean-capital version: lab block D.

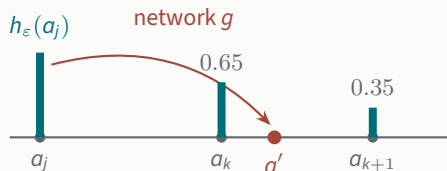
Encoding the Distribution: Mass on a Grid of Points



- ▶ N_b equidistant points: the first is the limit $\underline{a}$; the last high enough that little mass reaches it. Savings above it are set to it.
- ▶ The code: 100 points on $[0, 20]$, $\Delta \approx 0.2$. A mass $h(a_j)$ sits on each point: a finite vector the network can read, and move exactly.
- ▶ **Unlike DeepHAM: the distribution is written on a grid, not sampled by 50 households.**

Source: The paper, following Reiter (2009) and Young (2010); the continuum-of-agents code (the asset grid).

From Today's Histogram to Tomorrow's: One Lottery Step



For each ϵ and grid point a_j

- 1 saving $a' = g(\epsilon, a_j; X)$, set to a_{N_b} if above
- 2 a' lies between a_k and a_{k+1}
- 3 weights $w_k = 1 - |a_k - a'|/\Delta$, $w_{k+1} = 1 - w_k$
- 4 send $w_k h_\epsilon(a_j)$ to a_k , $w_{k+1} h_\epsilon(a_j)$ to a_{k+1}

$$h'_{\epsilon'}(a_k) = \sum_{\epsilon} \pi(\epsilon' | \epsilon; z, z') \sum_j h_\epsilon(a_j) w_k(a'_{\epsilon j})$$

- ▶ Then employment moves between the two histograms, with $\pi(\epsilon' | \epsilon; z, z')$: one h' for each z' . Mass is conserved; the mean of a' is exact.
- ▶ **Unlike DeepHAM: tomorrow's distribution is computed, 200 network calls, not simulated household by household.**

Source: The continuum-of-agents code (the grid weights and the next-period histogram); Young, *JEDC* (2010).

The Equilibrium Conditions Are the Loss

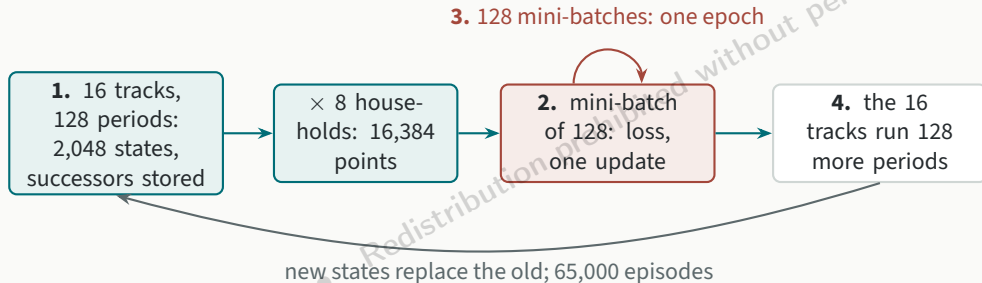
- At a point $(\varepsilon, a; X)$, with $c = (1 - \delta + r)a + y(\varepsilon) - a'$ and c' from the same network at $(\varepsilon', a'; X')$:

$$R_{EE} = \frac{\left[\beta \mathbb{E}[(1 - \delta + r')/c'] + \lambda \right]^{-1}}{c} - 1, \quad R_{KKT} = a' \lambda.$$

- $\mathbb{E}$: an exact sum over the four (z', ε') . $R_{EE} = 0$ is the Euler equation; $R_{KKT} = 0$ lets $\lambda > 0$ only at the limit.
- $X' = (z', h'_u, h'_e)$ is the lottery step of the last page, one for each z' .
- **Unlike DeepHAM: first-order conditions one period ahead, not utility over a 150-period rollout.**

Source: The paper; the continuum-of-agents code (its Euler and KKT terms).

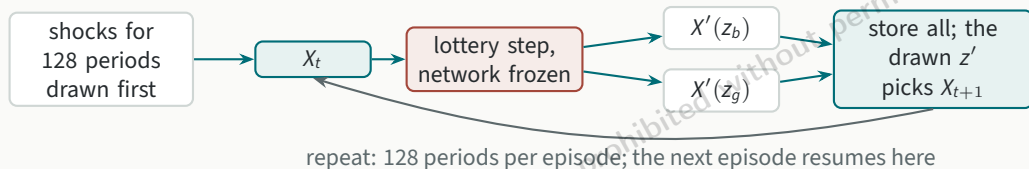
Deep Equilibrium Nets' Training Loop in One Picture



- ▶ **1. Samples:** 16 tracks $\times$ 128 periods, successors stored; 8 households per state.
- ▶ **2–4.** Shuffled mini-batches of 128; one epoch; then the tracks continue.
- ▶ **No look-ahead in the loss: the successors were stored in the simulation.**

Source: The continuum-of-agents code (the episode loop, training data, batching).

1. Samples: Simulated, and Stored With Their Successors



- ▶ **Simulation for data:** 16 tracks, 128 periods each per episode; the next page shows where they start and how they continue.
- ▶ Each period: one lottery step per z' ; all successors stored; the pre-drawn shock picks the next state.
- ▶ **Inside the loss:** no simulation; $t + 1$ is a stored successor.
- ▶ **A one-period condition at states from a long simulation; DeepHAM simulates 150 periods inside its loss.**

Source: The continuum-of-agents code (shock simulation, the forward pass, the episode loop).

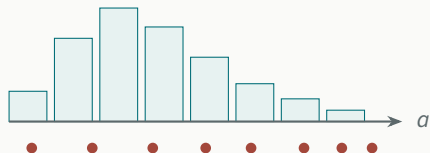
1. Samples: 16 Tracks, 128 Periods an Episode



- ▶ Episode 1: 16 copies of *one* fixed state (wealth near zero), differing only by their shock draws.
- ▶ Each episode: 128 periods per track, network frozen, $16 \times 128 = 2,048$ states; then train. At each dashed line, every track continues from its own end state, under the updated network.
- ▶ **Not 16 hand-picked starts: 16 long simulations, in 128-period pieces.**

Source: The continuum-of-agents code (starting states, shock simulation, the episode loop). The paths are schematic.

1. Samples: Each State Paired With 8 Households



one stored $X = (z, \mu)$, paired
with 8 random household
states: 8 evaluation points

- **Data:** the $16 \times 128 = 2,048$ aggregate states just simulated, each with its stored successors.
 - **Evaluation points:** each state X is paired with 8 household states (ε, a) , drawn at random: a uniform on $[0, 20]$, ε a fair coin. 16,384 points $(\varepsilon, a; X)$.
- X is not a starting point: nothing is simulated from it. It is where the loss is evaluated, with its stored successors $X'(z')$.
- Why points: the network is pointwise, $(\varepsilon, a; X) \mapsto (a', \lambda)$, and the Euler condition is a household's. Why uniform: the lottery uses the policy at all 200 grid points.

Source: The continuum-of-agents code (training-data construction).

2-5. Mini-Batches, One Epoch, and the Loss

- ▶ **Mini-batches**: the 16,384 points are shuffled and cut into 128 mini-batches B of 128. **Inside one update**, each point gives one Euler and one KKT residual:

$$\mathcal{L}(\theta; B) = \frac{1}{128} \sum_{(\epsilon, a; X) \in B} \left[R_{EE}^2 + R_{KKT}^2 \right], \quad \text{one Adam step at rate } 10^{-5}.$$

- ▶ The successors X' were stored during simulation: the network is re-evaluated at them, but how the histogram moves is not differentiated.
- ▶ **Epoch**: the 128 mini-batches use every point once; then the next episode, the 16 tracks run 128 more periods.
- ▶ One period ahead only: the long simulation chooses the states; the loss never looks past $t + 1$.
- ▶ **Unlike DeepHAM: no rollout inside the loss and no critic; the gradient sees one period, at stored states.**

Source: The continuum-of-agents code (the loss and the training step).

Deep Equilibrium Nets: the Numbers, and What Each One Is

number	what it counts	where it appears
100 / 200	grid points / histogram masses (2×100)	the state X , with the shock
16	tracks (economies), identical at the start	each continues from its own end state
128	periods per economy per episode	$2,048/16$
2,048	aggregate states per episode	16×128 : the data of one episode
8	household points per state	random (ϵ, a) draws; 2×4 in the code
16,384	evaluation points per episode	$2,048 \times 8$
128 / 128	points per mini-batch / Adam steps per episode	$16,384/128$; one pass, each point once
65,000	episodes	about 8.3 million Adam steps in all, rate 10^{-5}

- ▶ All are settings of the public code: choices, not properties of the model.
- ▶ **One episode: simulate 2,048 states, pair them into 16,384 evaluation points, take 128 steps, and move on.**

Source: The continuum-of-agents code (run settings: batch size, episodes, episode length, economies, points per state, learning rate).

Deep Equilibrium Nets as an Algorithm

Deep Equilibrium Nets on Krusell–Smith

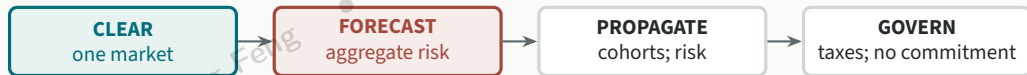
- 1 16 tracks, identical at the start: a shock and two 100-point histograms
 - 2 for each of 65,000 episodes (a setting of the code):
 - 3 draw 128 shocks; each period one lottery step per z' , store all, move to the drawn one
 - 4 store the $16 \times 128 = 2,048$ states with successors; pair each with 8 random (ε, a)
 - 5 one pass of 128 batches: Euler and KKT residuals, one Adam step each
 - 6 each track continues from its own last state in the next episode
- ▶ Intuition: if the first-order conditions hold at every state the economy visits, the policy is an equilibrium; the histogram makes the state exact.
 - ▶ **Two levers against DeepHAM: an exact state instead of a sampled one, and a one-period condition instead of a long rollout.**

Source: The paper; the continuum-of-agents code.

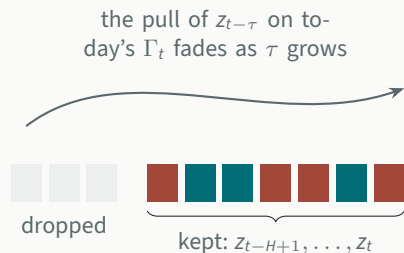
Session 4.2

Method 3: The Sequence Space

Forget the distribution: the recent shocks already determine it



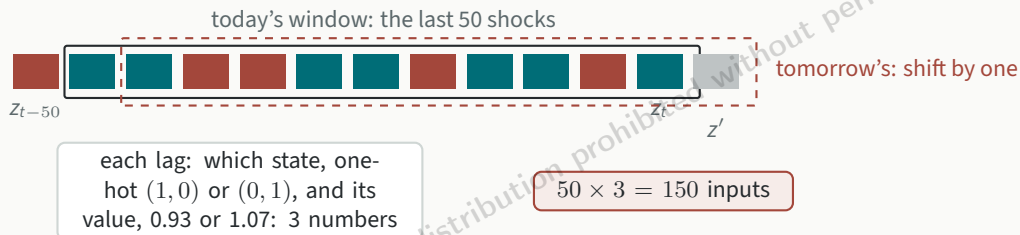
The Distribution Is a Function of Past Shocks



- ▶ Given the start and the policy, today's distribution is a function of all past shocks: $V(a, \varepsilon; \Gamma_t, z_t) = V(a, \varepsilon; z_0, \dots, z_t)$.
- ▶ **Memory loss:** old shocks fade, so keep the last H : $V \approx V(a, \varepsilon; z_{t-H+1}, \dots, z_t)$.
- ▶ The input is *exogenous*: its distribution is set by the shock process, whatever the policy does.
- ▶ **Why it works: history $\rightarrow \Gamma$ is the law of motion iterated under one fixed policy. The tutorial keeps $H = 50$.**

Source: Method 3 follows Azinovic-Yang & Žemlička, “Deep Learning in the Sequence Space”, working paper, arXiv 2509.13623 (2025), with their slides (2026) and tutorial notebook; that notebook in PyTorch: the Session 4 lab, block E.

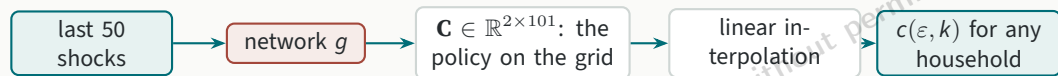
Encoding the Input: a Window of 50 Shocks



- ▶ Stored as all 50 one-hot pairs, then all 50 values: 150 numbers that stand in for the aggregate state, the distribution included. For tomorrow, drop the oldest shock and append z' .
- ▶ **Unlike DeepHAM: no household state and no moment enters; the input is exogenous, its distribution fixed by the shock process.**

Source: The tutorial notebook (history encoding and rolling windows).

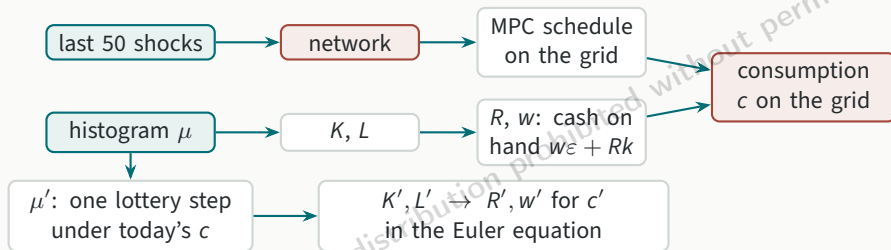
Operator Learning: History In, a Whole Policy Out



- ▶ **Operator learning** (Zhong, 2023; the paper extends it): write the policy as $f(X, x) = g(X)(x)$. The network g maps the aggregate input, here the history, to a whole *function* of the individual state $x = (\epsilon, k)$.
- ▶ The function is stored as its values on a fixed grid, and read between grid points by linear interpolation. One call gives every household's policy.
- ▶ Shape is imposed at the grid points: the output is a starting share $\alpha(\epsilon) < 1$ and falling MPCs, so c is increasing, concave and leaves $k' \geq 0$; linear interpolation keeps that shape between points.
- ▶ **Unlike DeepHAM, whose network is evaluated household by household: here the network's output is the policy function itself.**

Source: Zhong, Y. (2023), "Operator Learning in Macroeconomics", working paper; the tutorial notebook.

Where the Distribution Still Enters: Through Prices



- ▶ The network reads only shocks and gives the *shape* of consumption; the *level* needs today's prices, from μ . Tomorrow's prices need μ' , one lottery step on.
- ▶ So every state is a pair (last 50 shocks, μ), with μ simulated along those same shocks. Even a one-period loss needs a starting μ .
- ▶ **The network reads shocks; the distribution sets the prices it is evaluated at.**

Source: The tutorial notebook (the network's forward pass takes the history and today's prices; the loss).

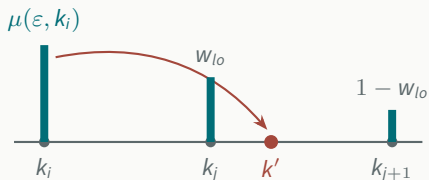
Why a Log-Spaced Grid



- ▶ The grid is where the policy function lives. Linear interpolation errs where the function bends, and consumption bends most near the borrowing limit.
- ▶ So points are dense there: equally spaced in $\log(k + 0.5)$, 0.03 apart near zero, 5 at the top. The paper suggests log-log spacing, for the same reason.
- ▶ **The histogram sits on the same grid: the lottery moves mass straight from the policy's own grid values.**

Source: The paper's section on operator learning; the tutorial notebook (capital grid).

The Same Lottery, on the Log Grid



For each ϵ and grid point k_i

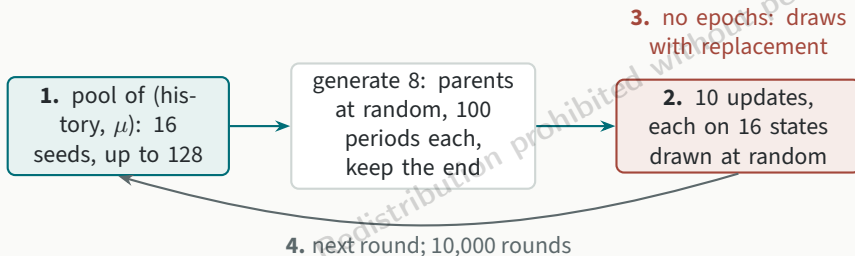
- 1 $k' = w\epsilon + Rk_i - c(\epsilon, k_i)$, clipped to $[0, 100]$
- 2 bracket: $j = \lfloor (\log(k' + 0.5) - x_0) / \Delta x \rfloor$
- 3 $w_{lo} = (k_{j+1} - k') / (k_{j+1} - k_j)$
- 4 send $w_{lo}\mu$ to k_j , the rest to k_{j+1}

$$\mu' = \Pi_{\epsilon}^{\top} \mu_{\text{after}}, \quad K' = \sum_{\epsilon, k} k \mu'(\epsilon, k), \quad L' = \sum_{\epsilon, k} \epsilon \mu'(\epsilon, k).$$

- Identical in spirit to Method 2's step; only the grid differs. In the tutorial, Π_{ϵ} does not depend on z .
- **What differs from Deep Equilibrium Nets is who reads μ' : there the network does; here only K' and L' do.**

Source: The tutorial notebook (the distribution step); Young, *JEDC* (2010).

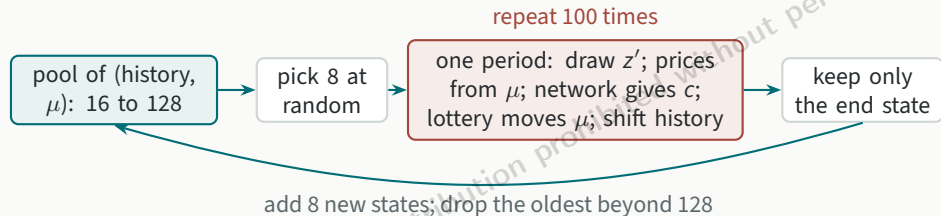
The Sequence Space's Training Loop in One Picture



- ▶ **1. Samples:** each round adds 8 states, run 100 periods from random parents.
- ▶ **2-4.** Each update draws 16 at random; no epochs; 10,000 rounds.
- ▶ **The loss looks one period ahead (both z'): it moves nothing in the pool.**

Source: The tutorial notebook (buffer, parent draws, training loop).

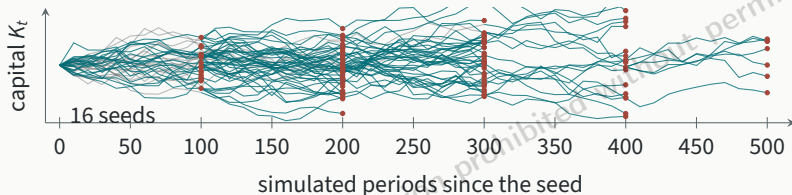
1. Samples: One Round Adds 8 States



- ▶ A state is a pair (last 50 shocks, μ); the 16 seeds are one all-low history with μ_0 . Each round, 8 states run 100 periods, network frozen; only the end state is kept.
- ▶ One round: generate 8 states, then 10 gradient steps on 16 states drawn from the pool. Pool size: 16, then 24, ..., 128 from round 14.
- ▶ **Unlike DeepHAM: 128 states in all, each a whole economy, renewed 8 at a time.**

Source: The tutorial notebook (seed, continuation, pool update).

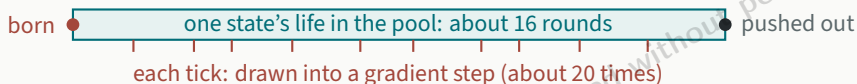
1. Samples: a Branching Pool



- ▶ Each round: 8 parents drawn at random, with replacement; each child is its parent run 100 more periods on fresh shocks. Red: the pool after 20 rounds; grey: pushed out.
- ▶ Replaying the rule for 10,000 rounds: the pool lies about 120,000 simulated periods from the seed.
- ▶ **Method 2 runs 16 fixed tracks; here tracks split and die out, and the pool is a tree's tips.**

Source: The tutorial notebook (buffer seeding, parent draws, continuation, pool update); the depth is a replay of that rule. Paths schematic.

2-4. Mini-Batches Drawn at Random; No Epochs; 10,000 Rounds



- ▶ **Mini-batch:** each update draws 16 states at random, with replacement, from the pool as it stands.
- ▶ **No epochs:** nothing is used up in order. Each round makes 10 updates; a state is drawn about 20 times in its 16 or so rounds (above).
- ▶ **Rounds:** generate 8, then 10 updates. Over 10,000 rounds: 80,000 states from 8 million simulated periods, 128 held at a time, 1.6 million state-draws.
- ▶ **Every training state was simulated under a recent version of the network.**

Source: The tutorial notebook (training loop). The paper instead moves 4,096 pairs one period per episode.

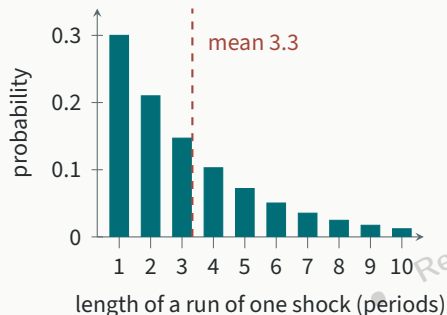
5. Inside One Update, Line by Line

One Adam step

- 1 draw 16 states (history, μ) at random from the pool built on the previous pages
 - 2 for each: R, w from μ ; one call $\mathbf{C} = g(\text{history}; R, w) \in \mathbb{R}^{2 \times 101}$; $k'_{ij} = w\varepsilon_i + Rk_j - C_{ij}$
 - 3 $\mu' =$ one lottery step of μ under c ; then K', L' , held fixed
 - 4 for each z' : $\mathbf{C}' = g(\text{shifted history}; R', w')$, read at each k'_{ij} by interpolation
 - 5 $q = \sum_{z', \varepsilon'} \pi \beta R' / c'$; then the Euler gap $e = 1/(qc) - 1$, $s = k' / c$, $\text{FB}(e, s)$
 - 6 loss = mean of FB^2 over 16×202 ; one Adam step (rate 5×10^{-5} , clipped at 1)
- ▶ One network, one call per state: its output is the whole consumption function on the grid, $c(\varepsilon_i, k_j) = C_{ij}$; not a different network per node.
 - ▶ The gradient reaches the network through $\mathbf{C}$ and $\mathbf{C}'$ only; prices, μ and μ' are held fixed.
 - ▶ **Unlike DeepHAM: one period, every grid node of every state, no rollout and no critic.**

Source: The tutorial notebook (the loss and the training step).

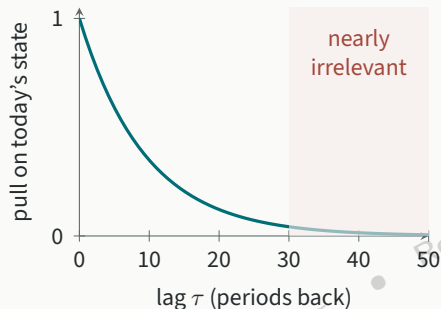
Coverage (1): The Draws Follow the True Shock Process



- ▶ All $2^{50} \approx 10^{15}$ histories are possible, and none is ruled out.
- ▶ But each new shock is drawn from the true chain: stay with probability 0.7, switch with 0.3. Training therefore meets histories as often as the economy does.
- ▶ Runs last $1/0.3 \approx 3.3$ periods on average, so likely histories are a small, structured part of the space.
- ▶ **Unlike DeepHAM and Method 2, the history's distribution does not depend on the network; only μ does.**

Source: The tutorial notebook (shock process and training loop).

Coverage (2): Old Shocks Matter Little, and μ Comes Along



- **Memory loss:** an old shock's pull on today's state decays geometrically (in the paper's one-sector example, at rate $\max\{|\rho|, \alpha\}$). Histories that differ only far back need nearly the same answer.
- μ needs no coverage of its own: it is whatever the simulated history produced.

- **Caveats:** each μ in the pool was simulated under an older network; coverage is checked on simulated paths, not guaranteed; a pool of 128 is a teaching setting.

Source: The tutorial notebook; the paper's argument that a truncated history is approximately sufficient. The decay curve is schematic.

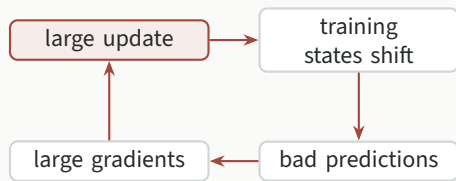
Sequence Space: the Numbers, and What Each One Is

number	what it counts	where it appears
50 / 150	shocks in the history / network inputs	the network's input: 3 numbers per shock
101	asset grid points (log-spaced)	the policy's values and the histogram share them
$2 \times 101 = 202$	grid nodes (ε, k)	one Euler residual each, per state
16	seed states	the pool at the start: all-low history, μ_0
128	pool capacity	the oldest state leaves when it is full
8 / 100	states continued per round / periods each	data generation: only the end state is kept
16	states per gradient step	drawn at random from the pool
10	gradient steps per round	after the 8 new states arrive
10,000	rounds	191 CPU-seconds in all

- **One round: generate 8 states (100 periods each), then 10 steps of 16 states $\times$ 202 residuals.**

Source: The tutorial notebook (settings, training loop, stored output).

What the Exogenous Domain Buys



state-space training can feed on itself; a history's distribution does not move with the policy

- ▶ Reported: stable training across learning rates and seeds, where the state-space version struggles.
- ▶ Tutorial: mean $|\text{FB}| 1.6 \times 10^{-4}$. Its economy: annual; ϵ independent of z .
- ▶ The cost: the policy cannot be asked about a distribution no history produced.
- ▶ **Unlike the other two: the inputs do not shift when the policy does.**

Source: Stability: their slides (2026), pp. 11, 23–24. $|\text{FB}|$: the tutorial's stored output.

Session 4.2

Three Methods Against a Law of Motion

What each method moves forward, and what a government's problem needs



How Far Forward Each Method Moves the Distribution

	what moves Γ	horizon in the loss	gradient through the move?
DeepHAM	a panel of 50, simulated	150 periods	yes, through the sampled panel
Deep Equilibrium Nets	Young's lottery on a histogram	one period	no: successors are stored data
Sequence space	Young's lottery, beside the history	one period	no: prices and K' held fixed
Feng, Han, Sargent & Zhu (2026)	the lottery matrix $\hat{\Phi}(\mathbf{x}; \theta)$, in the graph	T : 100; 30 with a government	yes, through every $\hat{\Phi}_t$

- **Methods 2 and 3 use the one-period lottery only to generate data; Feng, Han, Sargent & Zhu (2026) differentiate the whole path, DeepHAM through a sample.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; the three sources' code, as on the previous pages.

Argument 1 (a): What a Markov Government Has to Compute

- ▶ Each period the government picks a tax τ ; households respond with g_τ ; successors follow the rule $\tau(\cdot)$:

$$V^G(\Gamma, z) = \max_{\tau} \left\{ W(\Gamma, z, \tau) + \beta \mathbb{E}[V^G(\Gamma', z') | z] \right\}, \quad \Gamma' = H(\Gamma, z, z'; g_\tau),$$

and in equilibrium the maximiser is the rule itself: $\tau(\Gamma, z)$.

- ▶ Its first-order condition:

$$\frac{\partial W}{\partial \tau} + \beta \mathbb{E} \left[\frac{\partial V^G}{\partial \Gamma'} \cdot \frac{\partial H}{\partial g} \cdot \frac{\partial g_\tau}{\partial \tau} \right] = 0.$$

- ▶ H is the law of motion, not a histogram: it maps today's distribution to tomorrow's. The second term: τ changes saving g_τ , which moves Γ' through H ; with $\beta R \approx 1$ it compounds.
- ▶ **Needed: a rule that reads Γ , households that respond to τ , and $d\Gamma'/d\tau$ through H .**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; Klein, Krusell & Ríos-Rull, *RES* (2008).

Argument 1 (b): Which Method Supplies What

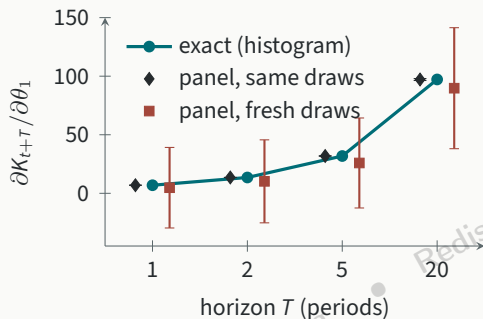
	rule reads Γ	households respond to τ	$d\Gamma'/d\tau$	successor governments
DeepHAM	through a moment of 50	tax fixed; a planner option picks c	through a sampled panel	no
Deep Eq. Nets	yes, the histogram	not in the code	no: Γ' is stored data	no
Sequence space	no: shocks only	not in the code	no: prices held fixed	no
Feng, Han, Sargent & Zhu (2026)	yes, the histogram	yes: implementability	yes: through $\hat{\Phi}$, every date	yes: a perceived rule

Feng, Han, Sargent & Zhu (2026) so far without aggregate risk: adding z is open.

- The sequence space fails at the root: history $\rightarrow \Gamma$ holds under *one fixed* policy; a deviating government breaks it. **Only a law of motion in the graph supplies the second term of the government's condition.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; Han, Yang & E, arXiv 2112.14377, §2.4, §5; the three sources' code.

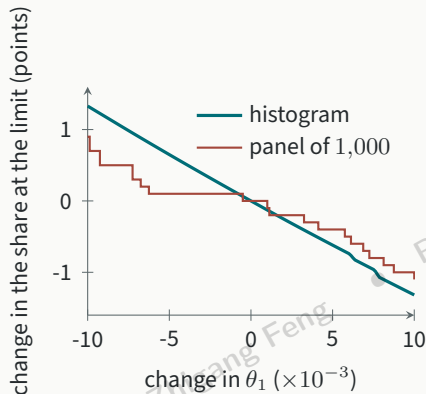
Argument 2 (a): A Derivative From a Panel, and From the Histogram



- ▶ **The test:** households save $a' = \theta_0 + \theta_1$ cash, clipped at the limit. How far does capital T periods on move per unit of θ_1 ?
- ▶ **Exact**, through the histogram: 6.90 at $T = 1$.
- ▶ **A panel** of 10,000: simulate at $\theta_1 \pm 0.001$ and difference. Fresh draws: 4.8 ± 34.4 ; same draws: 6.90 ± 0.05 . Mean $\pm$ one s.d. over 20 repetitions, as the bars.
- ▶ **For capital a panel works if its draws are fixed, as in DeepHAM's gradient.**

Source: Lab block B: $\theta_1 = 0.938$, a linear rule fitted to the solved policy; the lab's annual economy.

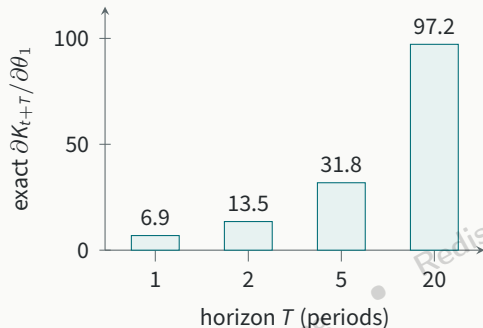
Argument 2 (b): Where a Panel Breaks: Head Counts and Clamps



- ▶ Same test, now the share at the borrowing limit after 20 periods; one household is 0.1 point.
- ▶ A step's derivative is zero wherever it exists. At $T = 2$: exact -0.13 ; panel 0.00 ± 0.00 .
- ▶ A clamped household ignores θ_1 : a panel gradient misses who is pushed onto the limit.
- ▶ **Only the lottery in the graph, moving mass $h da' / \Delta$, gives the whole derivative.**

Source: Lab block B's test swept over θ_1 , same draws at every slope; the lab's annual economy. Feng, Han, Sargent & Zhu (September 2026), preliminary.

Argument 2 (c): Why the Derivative Must Look Many Periods Ahead



- ▶ At $T = 20$ the effect is 14 times its one-period size, and still rising; five periods capture 33% of it.
- ▶ With $\beta R \approx 1$ an effect lives for about $1/(1 - \beta R)$ periods: about 420 in Deck 4.1's economy.
- ▶ A one-period condition (Methods 2 and 3) sees only the first bar.
- ▶ **DeepHAM sees 150 periods, through a sample; Feng, Han, Sargent & Zhu (2026), 100 or 30, exactly.**

Source: Lab block B, the exact histogram derivative; the lab's annual economy. Feng, Han, Sargent & Zhu (September 2026), preliminary.

Bridge: Each Tool for Its Question

Sequence space: long memory, stable training, an exogenous domain.

Deep Equilibrium Nets: the whole histogram as state; first-order conditions.

DeepHAM: interpretable moments; long rollouts.

Feng, Han, Sargent & Zhu (2026): the law of motion in the graph; a government without commitment.

- ▶ For the competitive equilibrium of this economy, all three work, and the classical rule is already accurate.
- ▶ The difference opens when someone chooses a policy that moves the distribution, and must be consistent with later choices.
- ▶ **Deck 4.3: households that age, and aggregate risk over cohorts, where the third method earns its place.**

Source: Feng, Han, Sargent & Zhu (September 2026), preliminary; the sources of this deck.